Reference-Preserving Speculative Decoding on Consumer Systems: Diagnosis, Repair Scope, and Paid Cost

Author information pending confirmation

Abstract

Speculative decoding can change a quantized runtime's greedy output as well as its cost. When a fixed ordinary output must be reproduced, a runtime designer must decide where to constrain execution and whether the qualified transformation pays. We study these decisions through retained-state interventions, a phase-scoped attention implementation, and paid ordinary and matched draft-free comparisons. Natural state–current-recipe crosses reveal path effects and interactions on CUDA and Vulkan. A same-state single-block intervention changes downstream trajectories despite unchanged immediate decisions. On RTX, mapping attention queries into the batch dimension preserves other batched computation; target prefill and continuation are inclusion-minimal within a tested four-switch family. A frozen recipe qualifies full responses on 24 project-unused inputs. Generation speed is 1.170 times ordinary and 1.169 times matched decoding. Resident-request speed is 1.742 times ordinary but 1.035 times matched: the draft-free target-path change already provides 1.683 times. Seven tasks regress against matched request cost, and one round reverses direction. Wider fixed-capacity drafting and a B570 endpoint repair qualify while losing performance. Historical M4 and 358H results limit the case for added control and draft-only optimization. The contribution is a concrete runtime design study linking local diagnosis, finite repair qualification, and paid cost, with failure boundaries retained. Repair evidence remains specific to the tested Qwen3 implementation family and configurations.

Keywords: speculative decoding, numerical consistency, execution state, runtime design, performance evaluation, consumer systems

1. Introduction

Speculative decoding accelerates autoregressive inference by proposing tokens with a draft and evaluating several positions with the target [1]. Its appeal on consumer systems is practical: additional intra-request work can amortize target execution when memory and traffic do not permit large inter-request batches. Yet replacing scalar continuation with speculative execution changes numerical call shapes, retained state, and bookkeeping. A mathematically valid sampling rule does not by itself certify that a particular quantized implementation reproduces a selected greedy output or lowers the cost of doing so.

We consider systems in which an ordinary greedy execution defines the required reference. Examples include regression testing and repeatable application behavior under a registered runtime configuration. The requirement is token or endpoint equality to that execution, rather than a claim that its answer is functionally correct or that every floating-point value is identical. This makes two engineering decisions inseparable. The designer must determine where execution consistency needs to be enforced, then measure whether the qualified transformation is useful. Constraining the wrong boundary can repair one input and introduce another deviation. Constraining all computation can recover output while losing the intended acceleration.

The boundary is not determined by the visible token prefix alone. Two executions can have emitted the same tokens while retaining different numerical key/value (KV) contents or pending state. Their next decision can depend on that history, on the current execution recipe, or on an interaction between them. Conversely, changing a score need not change the immediate greedy decision. An unchanged decision at one verification block can still precede different downstream output. These distinctions determine whether an implementation needs to constrain state formation, current continuation, or both.

The cost comparison has a related ambiguity. A repair can improve the target path before any drafting occurs. Comparing only repaired speculation with the ordinary reference assigns that path benefit to the entire deployment change. It does not isolate the value of speculation. A matched target-only path, using the repaired target execution without a draft, is therefore a second paid comparator. Neither comparator can be replaced by accepted-token fraction, target-call count, or isolated draft throughput.

Our cases show why both decisions matter. An RTX continuation intervention repairs one task and regresses another while preserving aggregate reference agreement. A later phase-scoped attention implementation qualifies full responses on 24 inputs unused in the accessible project history. Its resident-request speed is 1.742× ordinary decoding, but matched draft-free execution already gives 1.683×; speculation adds 1.035× on that path. A separate B570 implementation preserves the registered endpoint yet is substantially slower than both paid draft-free comparators. These outcomes support a system design study built around three questions:

1. **RQ1**, diagnosis: does a deviation follow retained numerical state, the current recipe, or their interaction?
2. **RQ2**, repair: within a defined implementation family,

which phase constraints are sufficient to preserve the reference?

3. **RQ3**, cost: when qualification holds, what benefit remains against ordinary decoding and against a matched target path?

The contributions are concrete. First, controlled natural-state crosses with own-path shams expose different local dependencies on CUDA and Vulkan; a separate B570 single-block intervention shows why immediate agreement cannot replace trajectory qualification. Second, an RTX query-layout transformation preserves batched projection and feed-forward work, with a four-switch factorial locating a sufficient target-phase scope in the tested family. Third, ordinary/matched/candidate costs and qualified counterexamples change the engineering decision about adding speculation, increasing draft width, retaining a guard, or optimizing only the draft.

These contributions concern an implemented runtime and finite qualification, building on established numerical-consistency and verification principles. Diagnosis remains an offline experimental procedure; the model-family and measurement limits are stated in Section 7.

Sections 2 and 3 define the runtime and contracts. Sections 4–6 answer the three questions. Section 7 states the resulting design decisions and limits. The independent supplement gives campaign details, historical results, failures and missingness; the author source package supplies arithmetic and figure reconstruction.

2. Runtime Architecture and Research Context

2.1. Execution as a stateful transaction

A speculative iteration advances a committed prefix through draft, target verification, and reconciliation. The draft proposes up to k tokens. The target evaluates the pending token and proposal positions, often through a caller batch containing several tokens. Acceptance, correction, and a possible bonus token determine what is committed. Rejected positions must then be removed or reconciled before the next iteration. Stopping is evaluated under the registered EOG and cap rule. Figure 1 locates the state interfaces and the four phase controls examined here.

The runtime state includes more than committed IDs. Target and draft KV ownership, logical positions, pending logits or tokens, sampler history, and loop counters connect successive calls. A pending token may already have been sampled and accepted into the sampler while remaining to be evaluated into the cache. A transfer that repeats or omits this event changes the transaction. Early intervention frontends therefore retain pending state and clone behavior as well as KV. The later B570 candidate instead binds phases in the same context and live sampler. The RTX cost candidate does not activate its optional state-transplant branch.

The microbatch capacity, denoted UB, limits how a caller batch is partitioned for backend execution. It is distinct from k, occupied query count, and the number of output rows reserved by the frontend. UB2 can process a caller with one occupied continuation token. A larger k can lower caller count while increasing the number of micrographs. Output reservation can change the recipe even at the same UB. These distinctions matter when interpreting a cross or a performance counter.

The native implementation is based on pinned llama.cpp releases and source-bound private frontends [2]. The late RTX graph uses Qwen3-8B as target and Qwen3-0.6B as draft [3], both Q8_0, with non-flash CUDA attention. The intervention uses existing ggml views and operations; it adds neither a new CUDA arithmetic kernel nor a new acceptance rule. B570 uses Vulkan with a different frontend and phase-binding operation. Metal and OpenVINO-related historical cases retain their own runtime contracts. These are software execution choices on consumer hardware, rather than controlled comparisons of hardware alone.

2.2. Reference preservation and prior determinism work

The output requirement is deliberately narrower than general determinism. A token sequence can match while logits or cache values differ. Conversely, deterministic repetition of a candidate can stably reproduce the wrong reference. We register the ordinary path and endpoint before asking whether a transformation preserves them. Numerical consistency research already explains why shape-dependent reductions can alter such behavior. Batch-invariant inference constrains reduction strategies across batches [4].

LLM-42 uses a decode–verify–rollback design, fixed verification shapes with padding, and replacement of decoded KV entries by verifier entries [5]. It also permits distinct consistent strategies for prefill and decode. Stage-Replay studies fixed-prefix precision controls and bidirectional cache transplantation [6]. Vosti specifies and verifies deterministic inference at a system level [7]. MarginGate uses margin-triggered verification and repairs confirmed mismatches [8]. These works establish principles that this manuscript uses rather than renames. Table 1 states the specific additional evidence and the comparisons not performed.

Natural crosses examine retained state and the current native recipe at a shared committed boundary. They motivate a candidate whose sufficiency requires native qualification. No same-contract implementation comparison with LLM-42, Stage-Replay or Vosti was performed.

2.3. Performance and heterogeneous execution

Heterogeneous placement and draft-budget control are established design directions. Dovetail and DuoDecoding exploit complementary CPU/GPU capabilities [11, 12]; TurboSpec and BanditSpec adapt speculation to execution conditions [13, 14]. The published HeteroSpec studies contextual heterogeneity for verification allocation [15]. Our title distinguishes the present reference-preservation question from that work.

Lossless but Not Free examines consumer backend cost, cache choreography, and cases in which verification fails to amortize target work [9]. Performance or Illusion? studies speculative decoding in a production engine and shows why apparent reductions in decoding work do not guarantee performance [10]. Here, output qualification, finite repair scope and ordinary versus

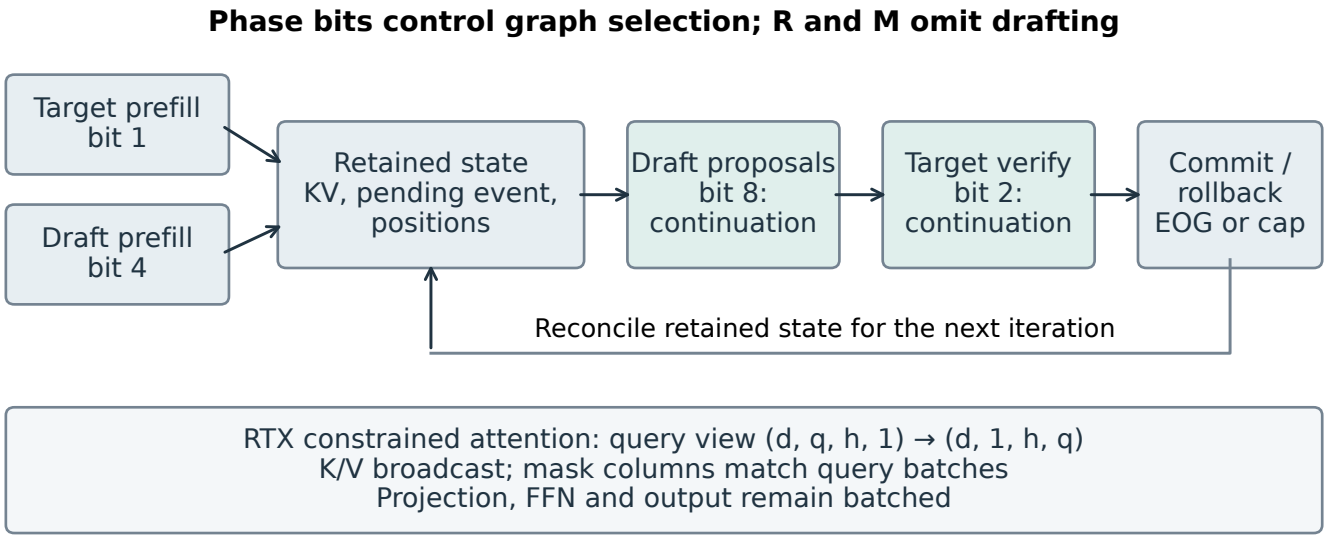


Figure 1: Stateful execution and four phase controls: target prefill/continuation (bits 1/2), draft prefill/continuation (bits 4/8). Commit/rollback reconciles retained state. Ordinary *R* and matched *M* omit drafting. The source-bound, ChatGPT/Codex-assisted diagram does not imply active state transplantation in the RTX cost candidate.

Table 1: Closest research context and the scope of this study. No row implies an implementation or speed advantage over the cited method.

Prior work	Established basis	Additional evidence here
Batch-invariant inference; LLM-42 [4, 5]	Reduction/shape consistency; verified tokens and KV reconciliation	Qwen3 query views, phase dispatch and finite scope, followed by ordinary and matched paid costs
Stage-Replay; Vosti [6, 7]	Fixed-prefix state intervention; specified system determinism	Natural donor × current recipe with shams; finite output qualification rather than a verified general guarantee
Consumer cost anatomy; Performance or Illusion? [9, 10]	Backend verification cost and weak performance proxies	Reference-qualified fixed-UB width counterexample and distinction between target-path and speculative request gains

matched paid baselines determine operating choices. Historical M4 and 358H results address control and offload limits under separate contracts; they do not replicate the RTX repair.

3. Study Design and Measurement Contracts

3.1. Systems, inputs, and evidence roles

Table 2 separates the principal contracts. All final targets are Qwen3-8B; changing device, draft representation, or backend does not create another target family. HumanEval [16] and sanitized MBPP [17] provide coding inputs. M4 also uses a prospective Spec-Bench partition [18]. Prompt construction and dataset identities remain campaign-specific. We measure reference reproduction, not benchmark test-pass accuracy.

The late RTX and B570 candidates use non-flash attention, greedy sampling and one request at a time. Their capacities, phase settings, model/runtime bindings and remaining parameters are frozen separately (Supplement §S1 and §S5.2). Campaigns are not pooled across contracts.

We distinguish five evidence roles. Prediction checks whether a source-bound harness reproduces native behavior. A selected-boundary cross tests a local dependence. Exposed probes select a finite repair. A frozen recipe is then checked on project-unused inputs. Finally, separately identified paid sessions assess qualified costs. Repeated runs assess stability and costs; forks and within-request decision events are correlated with their parent task. They are not additional independent inputs.

For the RTX expansion, input eligibility and the recipe were frozen before generating new references. Exposure accounting excluded previously executed or ambiguous records and selected 24 MBPP inputs from 224 eligible candidates. The intended unused HumanEval pool had no eligible inputs at that stage, so the pre-output amendment used MBPP. Project-unused means unused in accessible project records, not unseen during training. The measured panel becomes exposed for subsequent width and CPU analyses. B570's 15-input panel similarly follows its accessible-history audit; omitted RTX bulk records were not independently rescanned.

3.2. Registering the output requirement

Let $R(x)$ be the output of the paid ordinary path under fixed weights, tokenizer, formatting, sampler, runtime, backend, and endpoint. RTX expansion qualification checks the saved full response array, length, terminal ID and stop reason, together

Table 2: Principal study contracts. All cases process one request at a time. Historical text checks are listed separately in the supplement and are weaker than direct-ID qualification.

Case	Runtime/model setting	Output and exposure	Paid interval
RTX expansion	5070 Ti Laptop; CUDA; Q8_0 target/draft; private b10839-based frontend	Full response through EOG/cap128; 24 project-unused MBPP, four repeats	Server generation; resident HTTP request
B570 repair	Arc B570; Vulkan; Q8_0 target/draft; phase-binding frontend	First-EOG/cap128 endpoint; 15 project-unused inputs, two repeats	Native generation; whole request; host return
M4 runtime	MacBook Air M4; Metal; Q8_0 target/draft	Direct generated IDs; prospective 30 HumanEval plus 30 Spec-Bench, three repeats	Registered post-prefill runtime interval
358H stateful	Arc B390 Vulkan target; symmetric INT4 OpenVINO CPU/NPU draft	Direct IDs; 30 prompts, three repeats per draft condition	Registered stateful interval and target-work accounting

with reference repeat stability. All 384 formal rows belong to a complete $24 \times 4 \times 4$ task/condition/repetition matrix. Each condition has 96 qualified responses. References end at EOG for 23 tasks and at the cap for one; lengths range from 11 to 128.

B570 uses a different endpoint: retain IDs through the first runtime EOG or through token 128, whichever occurs first. Its speculative response can contain additional raw suffix IDs while satisfying this requirement. Those IDs and their work remain paid. We report endpoint equality and raw-array equality separately, including unknown historical raw comparisons. Historical re-tokenized-text agreement is retained in the supplement and is not retrospectively upgraded by later direct-ID experiments.

Qualification is necessary before a latency ratio is called a reference-preserving speedup. A shorter, divergent response does not qualify, even if its measured time is small. Repeat stability alone is insufficient: a stable candidate can consistently disagree with R. None of these finite gates proves functional correctness, equality of all logits, or preservation on arbitrary future inputs.

3.3. Controlled state and current-recipe crosses

At a naturally reached boundary, write the decision as

$$d = F(p, s, e), \tag{1}$$

where p is the committed prefix, s is retained numerical/runtime state, and e is the current recipe. Cross two naturally formed donors s_0, s_1 with two recipes e_0, e_1 while holding the registered prefix and current logical input fixed. A within-row change identifies a current-recipe effect for that donor; a within-column change identifies a donor-state effect for that recipe. An effect present under only one donor is a local interaction. Discrete equality does not imply equal continuous scores.

The cross requires valid cache ownership and positions, faithful state restoration, and compatible pending and sampler semantics. Own-path shams restore a donor under its own recipe and compare with its native behavior. Natural donors use online proposals rather than future reference answers or a recorded proposal stream fed into the candidate. Some earlier prediction experiments use independently sealed proposals for a distinct reproduction objective; Supplement §S2.1 keeps these roles separate. Selected boundaries deliberately probe known sensitivity, so their counts do not estimate prevalence among arbitrary requests. The available crosses also do not provide a complete counterfactual suffix for every cell.

3.4. Two paid baselines and task-level estimation

Define M as the repaired target path without a draft, and C as native speculation on that target path. R/C measures the full deployment change; M/C measures the increment from speculation; R/M measures the target-path change available without drafting. The ordinary R is an actually paid, qualified comparator, not a proven optimum over ordinary decoders.

For late RTX and B570, take each task's median absolute condition time before forming the baseline/candidate ratio. For n complete task pairs,

$$S_{b/c} = \exp\left[\frac{1}{n}\sum_{i=1}^{n}\log\left(t_{i,b}/t_{i,c}\right)\right]. \tag{2}$$

Values above one favor the candidate. Common tasks and aggregation imply

$$S_{R/C} = S_{R/M}S_{M/C}. \tag{3}$$

This is a ratio identity, not a component-cost decomposition. It cannot assign attention, KV, synchronization, or host fractions without independent profiling. M4 retains its frozen estimator and random streams; 358H takes the median of paired ratios before the task geometric mean, a different estimator.

The RTX expansion originally emphasized matched comparisons. Its ordinary-reference comparison is a later CPU reanalysis of the same paid records, not an additional predeclared hardware campaign. The reported descriptive intervals resample 24 task medians with NumPy PCG64, seed 20261008, 10,000 common index resamples and linear/type-7 quantiles. The supplement separately retains the earlier Python bootstrap convention. Task resampling does not account for session or day variation.

RTX generation is the source-bound server `predicted_ms` interval: draft/verification work between its timestamps is charged, but prompt work, the first-sample boundary and later final bookkeeping can lie outside it. Resident-request time surrounds non-streaming HTTP response receipt and parsing. Loading,

warmup, boundary telemetry and shutdown are excluded; asynchronous cleanup after response is not certified inside the interval. B570's native and host timers have separately documented boundaries. We do not pool unlike timers into one platform ranking.

The author-side CPU checks authenticate included sources and recompute exported arithmetic. Supplement §S7 distinguishes these checks from inherited original-host raw audits and identifies the access limits.

4. Locating the Consistency Boundary (RQ1)

4.1. A repair and a regression under the same intervention

An RTX intervention changes target continuation from UB2 to UB1 while retaining the registered prefill, draft and caller grouping. On 19 exposed HumanEval tasks, the source-bound predictor reproduces both native conditions on all 57 repeated complete arrays. Nevertheless, HumanEval/15 repairs, /25 regresses, and /18 remains divergent. Reference agreement stays at 51/57 in both conditions. An unchanged aggregate therefore hides opposing task-level effects; it is not an all-task no-op.

A separately selected 24-task MBPP panel has 72/72 prediction/native agreement in each condition but only 66/72 reference agreement. One persistent failure, MBPP/304, changes token 488 to 481 at zero-based position 33 while remaining a 47-ID divergent response. Prediction fidelity, response change, and reference restoration are three different outcomes. The continuation intervention alone fails the intended recovery-without-regression objective.

4.2. Natural donor state crossed with the current call

Four exposed HumanEval probes are examined at frozen natural cuts. Across repetitions, 24 donors support 48 state/current-path forks and 24 own-path shams. Donors retain actual pending state, cache positions and sampler behavior. Own-path decisions reproduce the original execution, and the original context remains unchanged after forking. Auxiliary short continuations and their 1,278 forwards are diagnostic work, not a performance panel.

Figure 2 shows discriminating IDs. At /15, with 13 committed tokens, changing the current UB2/UB1 recipe changes the third submitted decision from 9147 to 4199 under either donor. At /25, with four committed tokens, only the N1 donor/current-UB1 cell changes the submitted decision. That cell produces IDs 271,750,8168 rather than 1406. The effect is an interaction, not an explanation based only on donor state. At the selected /18 and /31 cuts, both recipes and donors preserve the discrete decision. These nulls are local and do not establish trajectory-wide invariance.

The RTX donor comparison also distinguishes logical tensor equality from file hashes. All 12 prompt-KV pair comparisons agree, while 70 of 72 generated-prefix K/V tensor comparisons differ; tested first-layer logical K/V remain equal. This supports examining state formation and attention-related execution, but does not identify attention as the sole cause. In particular, the discriminating /15 bonus row is itself a single-token microbatch under both histories. The final row's nominal shape cannot explain the entire effect without considering its inputs and preceding work.

4.3. One unchanged block does not certify the remaining trajectory

A separate B570 experiment starts both branches from the same live fork state, with byte-checked KV, and intervenes on one target verification block, comparing UB1 with UB128. The frozen checkpoint is the first eligible block reached after at least 64 committed IDs. Twenty-four reserved prompts with three repetitions provide 72 eligible forks. All 72 paired raw score records change, yet top-1 IDs and actual immediate decisions do not: zero of 72 executions and zero of 24 tasks show an immediate decision difference across 195 paired decision events.

Continuing under the frozen policy gives a different result. Complete downstream committed-ID trajectories change in 18 of 72 executions, corresponding to six of 24 tasks. Later alternate target blocks retain UB128 and proposals evolve, so the secondary result is not an isolated one-block perturbation followed by restored execution. The score effect and delayed trajectory effect coexist with an immediate discrete null. The intervention's primary sampled-block result therefore cannot be relabeled either an immediate decision effect or a whole-output no-effect result. Its engineering implication is direct: qualification must continue beyond a locally unchanged acceptance decision. Neither logit movement nor one-step agreement replaces complete registered-output checking.

4.4. B570 crosses identify a whole-recipe boundary

Later B570 natural-state crosses cover three tasks in two contexts, with 12 complete own-path shams and 24 auxiliary target cells. Restored states are checked for byte consistency, sequence-zero ownership and logical position. Figure 2 includes the selected argmax matrices. HumanEval/145 has $[(71465, 279), (279, 279)]$ at prefix 56, and /42 has $[(432, 1817), (1817, 1817)]$ at prefix 73. Both are local interactions. MBPP/72 has $[(320, 320), (308, 308)]$ at prefix 40: donor state changes the decision while the current columns are the same UB1/output-reservation-3 recipe.

For the HumanEval crosses, the current recipes change microbatch and output reservation together. The appropriate causal factor is the current whole recipe, not UB alone. The repeated cases are three tasks, not six independent task discoveries. Full counterfactual continuations from each cross cell are unavailable; the subsequent candidate has a separate native endpoint qualification panel. The greedy sampler contract is supported by live history, clone metadata and shams, while opaque sampler bytes and general stochastic-state restoration remain unavailable.

These observations carry the same experimental question across CUDA and Vulkan without transferring the same repair. Some boundaries are path-sensitive, others state-sensitive or interactive. A runtime designer cannot infer the required control placement from committed-prefix equality or a single margin alone.

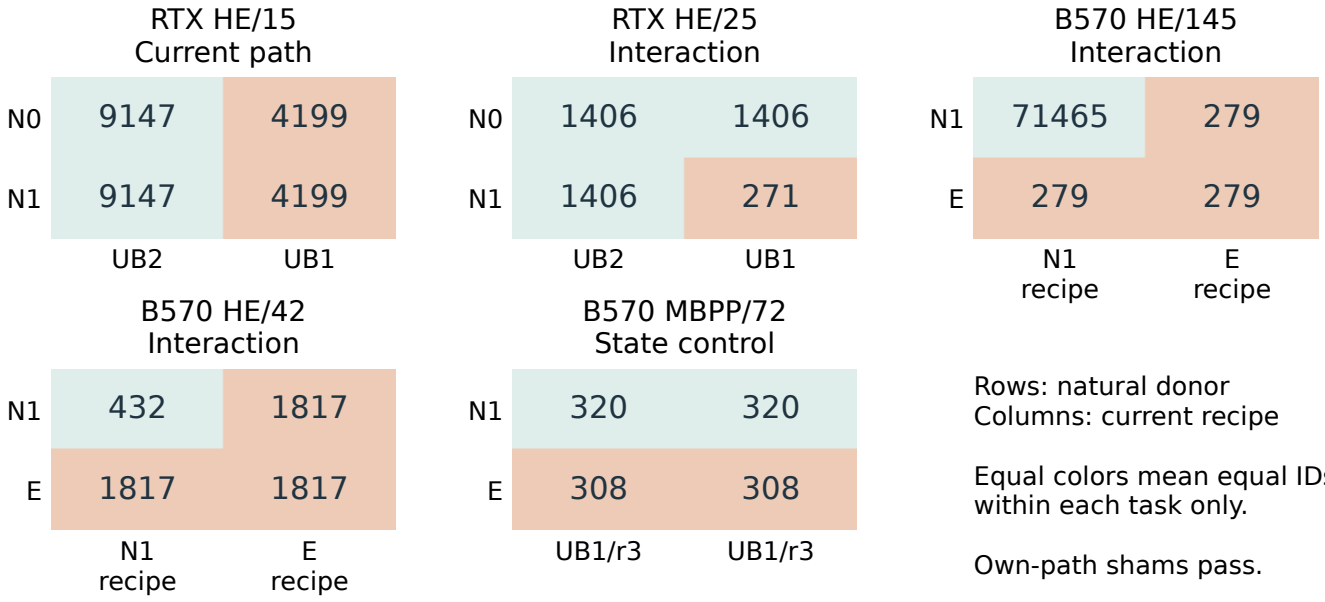


Figure 2: Source-derived RTX and B570 natural-state crosses. Rows are donors and columns current recipes. Cell labels are the discriminating sampled/argmax ID; colors group equal IDs within each task only. RTX /15 shows a current-path effect and /25 an interaction. B570 /145 and /42 cross N1 (UB1/reservation3) with E (UB2/reservation2); /72 has identical UB1/reservation3 current columns and is a state control. All repetitions reproduce the displayed local patterns.

4.5. Prediction and simple execution constrain added machinery

Earlier prediction studies remain useful but answer a narrower question. B570 prospective prediction matches 72/72 native arrays on a reserved 24-task cohort even when reference agreement is 60/72. RTX's first target-only batched-path predictor reaches 24/36, below its frozen 36/36 gate, so that study performs zero reserved inference. A later independent-proposal replay implementation obtains H2/native and reference-only/native prediction agreement of 72/72, with reference agreement still 60/72. The first implementation's missing interface is resolved by that later study; neither result makes replay a cheap online repair.

A fresh ordinary target-only bridge provides an especially strong simple alternative. On the 19 RTX tasks, draft-free U_2 reproduces all 57 intervention N_1 arrays and strict stop boundaries without a draft, transplant, proposal tape or answer. It reproduces both /15's repair and /25's regression. This is output equivalence, not demonstrated equivalence of unavailable live logits/KV or a speed result.

Finally, retrospective selector feasibility finds no additional supported capability: RTX direct enumeration and proxy screening both abstain at 114 post-common-acquisition record equivalents; B570 needs 72 for direct verification and 144 for proxy screening. Unknown counterfactuals remain unknown. These costs are acquisition equivalents, not measured latency. Together, the results answer RQ1 with local state/path evidence while rejecting the shortcut from an accurate reproducer to a useful automatic repair selector.

5. Phase-Scoped Implementation and Qualification (RQ2)

5.1. Constraining attention while retaining batched work

The RTX candidate changes the non-flash Qwen3 attention subgraph inside `build_attn_mha`. An initial implementation constructs one attention subgraph per query and concatenates the results. A subsequent implementation represents query columns in the ggml batch dimension, allowing existing operations to process them in parallel while retaining a one-column query structure. Projection, output projection and feed-forward computation remain batched. The change is local to the graph representation, rather than global UB1 execution of every operation.

After the existing query/key/value permutations, the query tensor has dimensions $(d, q, h, 1)$ in the supported single-KV-stream case. The candidate uses `ggml_view_4d` to reinterpret it as $(d, 1, h, q)$, preserving the source strides and using the original query-column stride for the new batch axis. The attention mask similarly changes from a query-column layout to $(n_{\mathrm{KV}}, 1, 1, q)$. Existing key-query multiplication, softmax, value multiplication and final contiguous recombination remain in the non-flash graph. Shared K/V tensors are not reconstructed from a reference answer. The purpose is to retain the tested scalar-query computation structure while preserving parallel execution elsewhere.

The source explicitly restricts this branch to Qwen3, non-flash attention, one KV stream, compatible mask dimensions, and absent attention bias, sinks and MLA value projection. The initial query limit is four; a source-bound later width build raises that assertion to eight without changing backend arithmetic. These guards define the implemented family. They do not establish

invariance for every model graph, precision, shape or backend. Native output qualification supplies the behavioral evidence.

5.2. Phase dispatch and graph reuse are part of the implementation

The scope mask is process-fixed. A thread-local guard selects one of four bits by model role and by whether the call is prefill or continuation: target bits 1/2 and draft bits 4/8. It restores the prior active state on exit. The guard executes before the capture-disabled fast path returns to native `llama_decode`, so disabling diagnostic metadata does not disable the constraint.

The selected boolean is copied into graph parameters and graph context. Graph reuse is rejected when that boolean differs from the cached topology key. This prevents a graph constructed for one phase layout from being reused solely because the ordinary microbatch fields match. The server also resets the phase to prefill when starting a new prompt. Scope selection is therefore an actual dispatch and reuse condition, rather than a label attached after execution.

Capture-disabled costs bypass observer metadata construction and forced observer synchronization. Qualification separately checks captured multi-query nodes and capture-on/off behavior. The source also contains a conditional target-state move after the first pending sample; the expansion environment does not set its activating continuation-UB variable. The measured RTX candidate consequently pays for native drafting, verification and reconciliation, but no state transplant is inferred merely from the presence of that branch. The supplement and author source package bind the relevant patch files and timer audit.

5.3. The finite phase family

Figure 3 presents the full four-switch factorial: 16 masks, four exposed HumanEval probes and two repetitions, totaling 128 requests. All outputs and registered stop boundaries are repeat-stable. Masks 3, 7, 11 and 15 pass the complete probe panel. /15 and /31 pass every mask; /25 needs at least one target-phase bit; /18 needs both target phases. Draft-phase bits do not change this pattern.

Mask 3 is the only passing set with no passing strict subset. This is inclusion-minimality in the registered family. The result neither proves that attention is the unique cause nor that draft behavior can be arbitrary in a different implementation. Acceptance, cache ownership, committed state and stopping still have to be valid. The $k = 3$/UB4 factorial also differs from the original UB2/UB1 intervention, so /15's all-mask agreement does not erase its earlier path effect.

Scope is selected by qualification, not the fastest measured time. A subsequent 19-task, six-repetition cost panel gives mask 3 1.278076× generation and 1.112977× request speed against matched target-only execution. Compared with mask 15, removing draft constraints adds only 1.003569× generation speed, with descriptive interval $[0.997739, 1.009456]$. It misses the frozen 1.03× extra-generation gate. Fewer constrained phases do not automatically improve speed. The later 24-input expansion similarly gives only 1.015426× extra generation benefit over mask 15.

5.4. Frozen inputs and geometry boundaries

The frozen mask-3/$k = 3$/UB4 recipe qualifies all 96 candidate responses on the 24-input expansion in Section 3. Matched target-only and mask-15 speculation also qualify. This extends sufficiency from exposed probes to a separately frozen input panel, while retaining the same model family and registered configuration.

Changing geometry exposes a failure boundary. An 11-input width/length panel separately tests $(k, \mathrm{UB}) = (1, 2), (3, 4), (5, 6), (7, 8)$ at caps 128/256/512 with fresh scalar references. The first two pairs qualify. MBPP/229 fails at both wider pairs across caps and repetitions, while its matched controls pass. A separate old /18 geometry probe also fails at wider target-only capacity and remains outside the 11-task denominator. Actual long reference lengths reach at most 261; a cap of 512 does not demonstrate a full 512-token trajectory.

At MBPP/229's common 47-token prefix, naturally retained constrained and ordinary states are crossed under two UB6 query layouts. Four own-path shams reproduce their decisions. Donor state changes the EOG decision under either layout; the current layout changes continuous values without a separate discrete flip in these cells. Two cap contexts repeat one task boundary. This supports a local retained-state effect and shows why switching only the current layout there is insufficient. It does not identify a unique cause of every width failure.

5.5. A separate B570 implementation and qualification scope

B570 provides a distinct repair case. Its candidate constructs target state with the ordinary-reference prompt caller and output reservation, samples pending state once, then changes continuation UB128 to UB1 in the same context and live sampler. A private API synchronizes, changes the capacity and requests scheduler reservation under a frozen context/batch/sequence contract. Instrumented checks compare serialized KV before and after binding; formal capture-off requests omit witness readbacks. The existing $k = 2$, draft-UB1 accept/bonus/rollback loop remains.

The candidate qualifies 12 development requests, 112 requests on 56 exposed inputs, and 30 requests on 15 project-unused inputs under its endpoint contract. It removes the old second context and three state witness readbacks, but does not implement the RTX query-layout transformation. Earlier B570 repair had a local long-panel positive result yet failed MBPP/72 on a new-input panel and failed five UB2 candidates. Those outcomes remain in the supplement. The later phase-binding candidate addresses qualification within its own scope; Section 6 assesses whether that repair pays.

6. Does the Qualified Design Pay? (RQ3)

6.1. RTX: deployment gain and speculative increment

The RTX expansion measures four conditions in four balanced rounds. Ordinary R uses mask 0/UB1 without draft; matched M uses mask 3/UB4 without draft; candidate C (also denoted C_3) uses mask 3/UB4/$k = 3$; C_{15} uses mask 15. Each round has four separate resident processes, each with one excluded warmup and

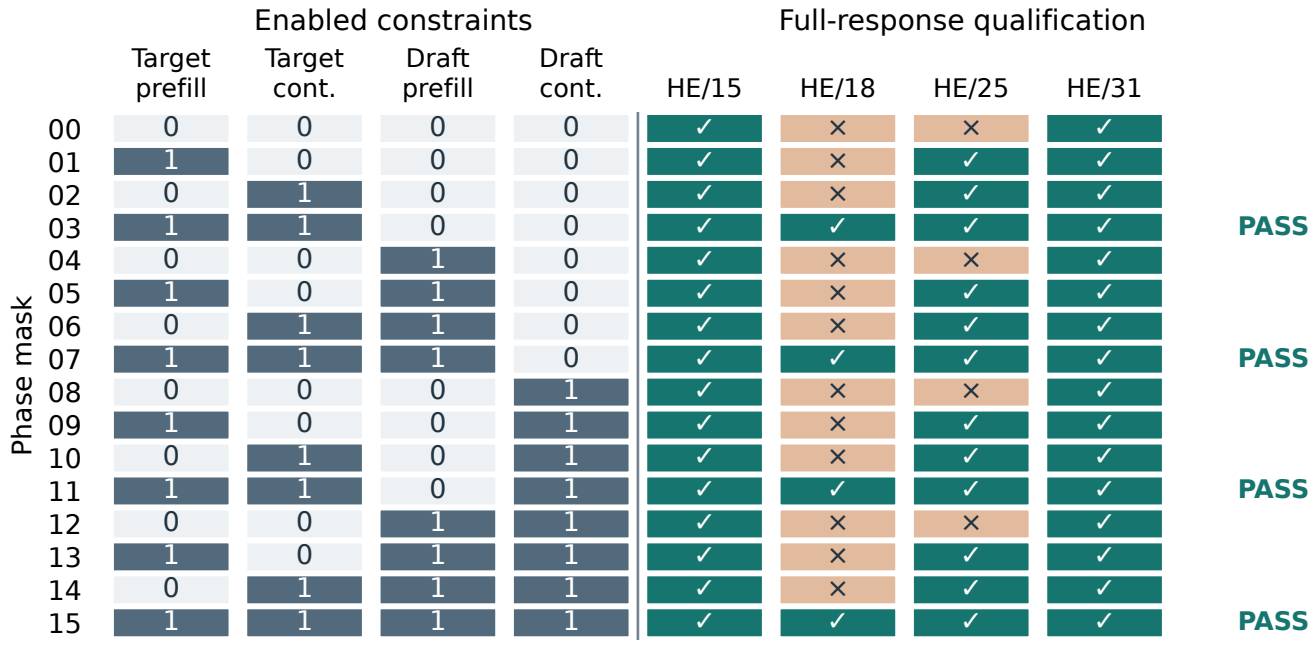


Figure 3: RTX phase qualification at fixed $k = 3$, UB4 and cap128. The 16 masks and four probes have two repetitions each (128 requests). Left columns mark phase bits; right columns mark native full-response qualification. Passing masks are 3/7/11/15. Mask 3 is inclusion-minimal within this switch family; it is not an operator-level or all-input minimality theorem.

24 formal requests. Condition and task orders rotate; recorded cache reuse is zero. Thus the paths share a panel and campaign but not one simultaneous resident process.

Table 3 reports the same-record CPU reconstruction against both paid baselines. Generation speed is 1.170124× ordinary and 1.169238× matched execution, with 22/24 tasks faster in each comparison. Resident-request speed is 1.741569× ordinary but 1.035071× matched. Crucially, R/M request speed is 1.682560×: most of the ordinary-reference request improvement is available without speculation. The 1.742× result is the benefit of the full deployment change and cannot be assigned entirely to speculative execution.

Table 3: RTX frozen expansion paid ratios: 24 task medians, four repeats. Gen. denotes generation; Request denotes resident request. Intervals describe task resampling. C_3 is mask 3 speculation.

Ratio	Timer	Speed	95% interval
R/C_3	Gen.	1.170124	[1.117,1.218]
R/C_3	Request	1.741569	[1.655,1.827]
M/C_3	Gen.	1.169238	[1.117,1.217]
M/C_3	Request	1.035071	[1.005,1.063]
R/M	Gen.	1.000758	[0.995,1.008]
R/M	Request	1.682560	[1.590,1.766]

Figure 4 displays the three comparisons on a common baseline/candidate axis while keeping generation and resident-request intervals distinct. All 48 task-level timer identities and both aggregate identities satisfy Equation 3; this check does not provide a component-cost profile.

The target-path comparison changes prefill micro-batch/attention behavior and continuation capacity jointly. Generation R/M is near parity, whereas the request ratio is much larger. Without independent profiling, that contrast cannot quantify attention, KV, transport, synchronization or host shares. It also cannot certify that R is the fastest ordinary decoder. A faster qualified ordinary implementation could narrow the deployment benefit without changing the arithmetic reported here.

6.2. *Task variation and session uncertainty*

The incremental request benefit is modest and uneven. Figure 5 keeps task generation and request ratios in the same request-sorted order. Seven tasks are slower than matched execution, although all 24 are faster than ordinary request cost. Matched-request round 2 reverses direction at 0.940220×; the other round geometric means are approximately 1.029, 1.052 and 1.044. The descriptive task interval for the median-based aggregate lies above one, but does not erase that reversal.

Separate resident sessions, laptop scheduling and thermal variation can affect these estimates. Balanced order and task medians reveal some variation but do not remove session confounding. A task bootstrap is not independent session/day replication. We therefore report a qualified positive aggregate operating point, rather than stable production request acceleration. The larger generation gain remains a result for its defined interval, not an end-to-end substitute.

The serial resident-reset study supplies a separate finite check: recorded requests begin with reset target state and the original $k = 3$/UB4 recipe retains reference output across two request orders. Its matched generation/request speeds are about 1.181 ×

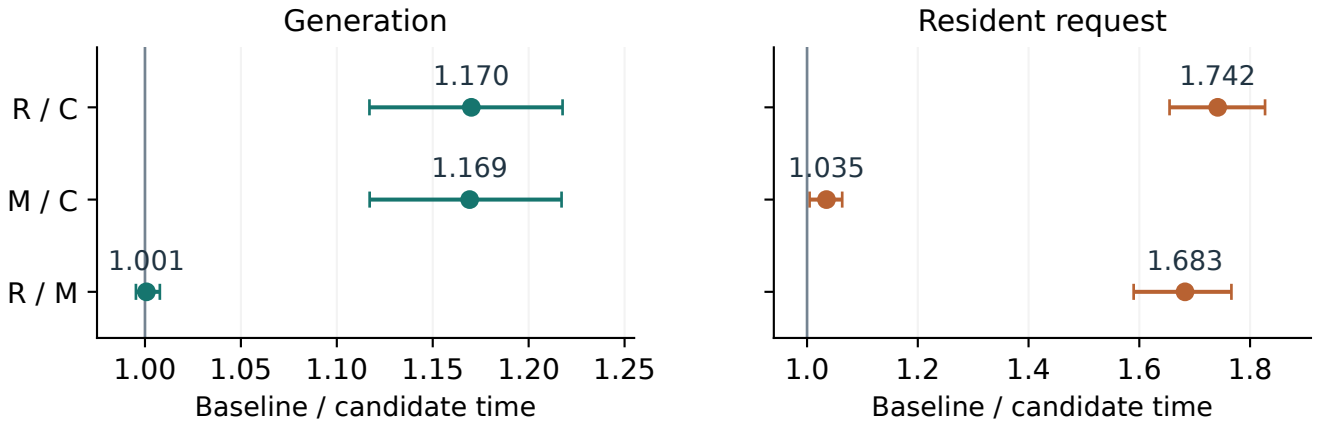


Figure 4: RTX paid speed ratios from 24 task medians, four repeats, with descriptive task-bootstrap intervals. Ordinary/candidate measures the deployment change; matched/candidate the increment from speculation; ordinary/matched the draft-free path change. The two timers cover different intervals. Ratios are not additive component costs.

/1.039×. This supports the registered reset behavior. It is not a concurrency or throughput experiment and is not pooled with the 384 expansion rows.

6.3. *Fixed UB: wider drafting qualifies but becomes slower*

The geometry failures in Section 5 change k and UB together. A separate frozen experiment holds UB4 and mask 3 fixed while testing $k = 3, 5, 7$ on 24 now-exposed MBPP inputs plus four exposed HumanEval probes. All widths qualify in two repetitions. The highest passing $k = 7$ is selected by qualification alone, then measured in separate capture-off sessions: four conditions, four repetitions and 448 formal requests. Qualification timings are excluded from speed selection.

Relative to the $k = 3$ anchor, $k = 7$ reaches only 0.829442× generation and 0.889868× resident-request speed. Against matched target-only, it reaches 0.997404 × /0.950353×, while the anchor reaches 1.202501 × /1.067970× in that campaign. The widened candidate preserves the reference and fails the extra-cost objective. Ordinary same-width speculation has output failures, so its times remain diagnostic rather than qualified speed evidence.

Formal draft work grows from 3,528 to 5,392 tokens while accepted draft tokens grow from 2,772 to 3,192. In separate captured qualification sessions, target continuation caller calls fall from 305 to 199, but observed micrographs rise from 305 to 397. These sessions include their own warmups and forced synchronization; the counts cannot be combined with formal times into a fabricated per-call profile. Micrographs are not CUDA kernel launches. The evidence rejects caller reduction as a sufficient performance proxy without assigning the slowdown to a unique component.

Saved partitions also matter. At $k = 7$, matched generation/request speed is 0.953/0.919× on 24 MBPP inputs but 1.308/1.164× on four HumanEval probes. The pooled result hides these differences. Four exposed probes do not validate a routing rule, and no online selector was developed from them. The supplement retains the longer-cap confirmation and the failed wider-geometry branches without combining their timers into an optimization trajectory.

6.4. *B570: repair improves an old path but loses to strong baselines*

The B570 formal matrix remains 119/120 complete: old N1's first-round MBPP/400 payload is missing after an infrastructure interruption. No request is imputed or rerun in the scientific denominator. Ordinary R, matched M and candidate C each have 30 complete responses, providing 15 two-round task-median pairs. Old-N1 comparisons have 14 complete task pairs.

Table 4 gives all three timers. Candidate speed is 0.282588× generation and 0.201290× whole request against ordinary R, and 0.285761 × /0.205704× against matched target-only. Both baselines are faster in every available paired round. The matched target path is close to ordinary cost (R/M = 0.988893 × /0.978539×), so the roughly fivefold candidate request cost cannot be attributed entirely to the consistency constraint on the target path.

Table 4: B570 paid baseline/candidate task-median ratios. n counts complete two-round task pairs: 15 for R/C, M/C and R/M, and 14 for old N1/C. Gen., Request and Host denote generation, whole request and host-observed time. The missing old-N1 task is not imputed; raw suffix work stays charged.

Ratio	n	Gen.	Request	Host
R/C	15	0.282588	0.201290	0.201884
M/C	15	0.285761	0.205704	0.206243
Old N1/C	14	3.993091	2.633940	2.633387
R/M	15	0.988893	0.978539	0.978864

Against old N1, the candidate improves generation/request speed by 3.993091 × /2.633940× over 14 task pairs. That is real engineering progress relative to a slow prior frontend, but does not establish deployment benefit. MBPP/245 also reverses

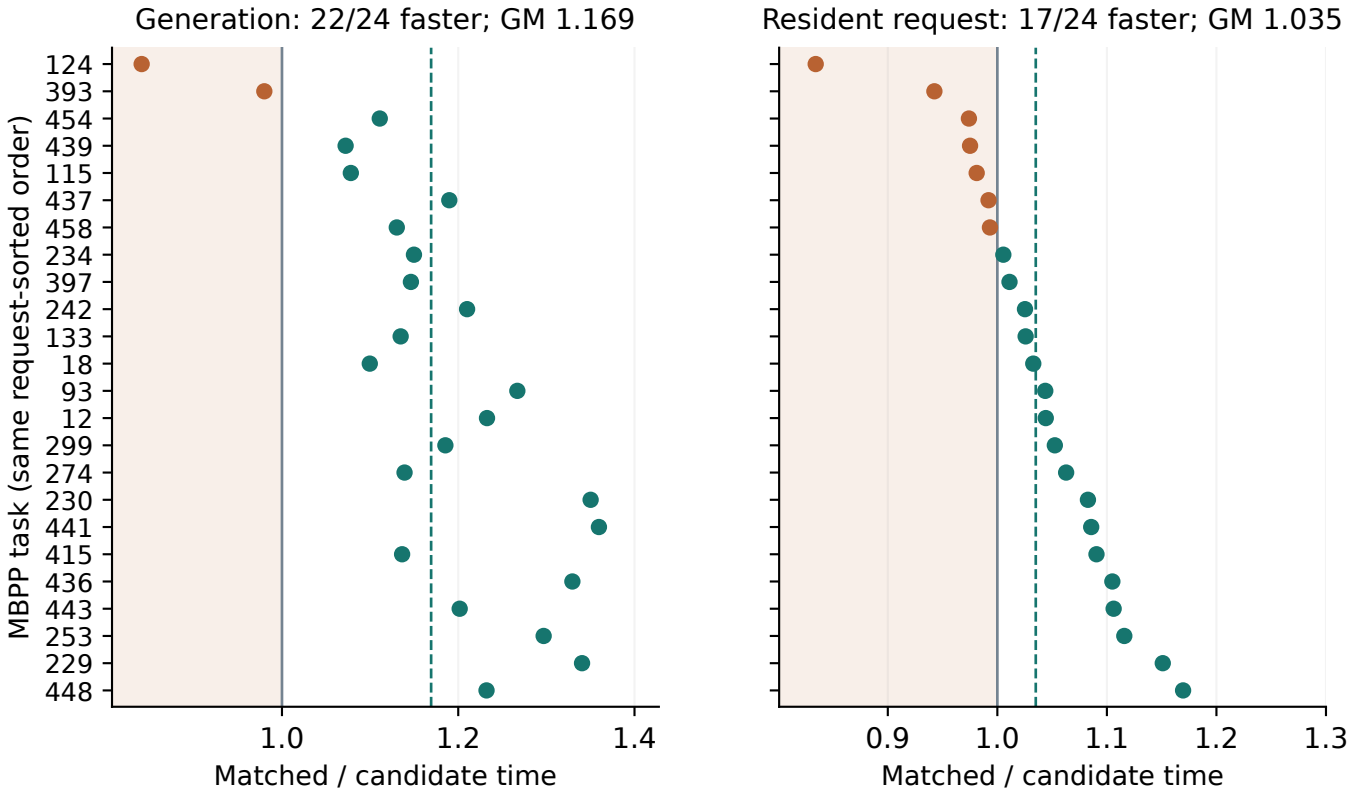


Figure 5: RTX matched/candidate task ratios from four-repetition absolute-time medians, in a shared order sorted by request ratio. Points represent tasks. Generation improves on 22/24 and resident request on 17/24; seven request regressions remain. Dashed lines mark aggregate geometric means and the solid parity line marks one.

the old-path request comparison in round 2. Neither aggregate improvement nor endpoint recovery supports a universal per-round gain.

The candidate's formal records contain 888 drafted and 702 accepted tokens. Endpoint IDs total 1,150 while raw IDs total 1,176; all 26 suffix IDs remain charged. Target and draft continuation caller-token totals are 1,332 and 1,362. These are distinct runtime quantities. Raw suffix equality is not required by this endpoint contract and is not claimed. Removing witness readbacks, an extra context and handoff improves the frontend, while phase reservation, synchronization, drafting and rollback remain online. No full independent component profile identifies a unique remaining bottleneck. The appropriate decision is to reject this recipe as a latency improvement against the two strong baselines.

6.5. Global serialization, control, and draft-side limits

An earlier RTX global-serialization timing arm qualifies 57/57 arrays on 19 exposed tasks but reaches only 0.677351× target-only speed, with descriptive interval approximately $[0.662, 0.692]$ and zero tasks faster. Its synchronized post-prefill-to-slot-release interval includes instrumented work and logging, unlike the later server generation field. It is a separate qualified cost failure, not the first point of a common before/after speed curve. The supplement preserves its host-timer amendment and the preceding V3 stop.

M4 addresses control under a separate post-prefill contract (Supplement §S6.6). A prospective 60-task, three-condition, three-repeat campaign has 540 requests; Always speculation and Guard each qualify 180/180 direct-ID comparisons. Target-only/Always speed is 1.346894×, whereas paired secondary Guard/Always latency is 1.027076 with descriptive interval $[1.012956, 1.043559]$. This guard is slower than the fixed policy. A hindsight oracle choosing only target-only or Always leaves about 0.069% further geometric-mean latency reduction; it does not bound intra-request policies.

On 358H, CPU/NPU drafting each qualifies a 90-request direct-ID panel but reaches only 0.733915 × /0.351663× speed. Holding measured target work and progress fixed, removing all other nonnegative work still gives prompt-estimator ceilings of 0.8372 × /0.8448×, below parity. Even this idealization requires a uniform target-work reduction of at least 16.28%/15.52% to reach parity. Faster isolated drafting cannot meet that budget; a different target path or state interface is needed. Supplement §S6.7 gives the conditional derivation, component totals and distinct pooled estimator. Neither M4 nor 358H is a cross-platform replication of the RTX repair.

7. Design Implications and Limits

7.1. Constrain the transaction where evidence warrants it

A current-path effect motivates controlling verification geometry; a retained-state effect motivates preserving state formation or changing its interface; an interaction requires checking both. The RTX crosses motivate target-phase attention constraints, whereas B570's distinct phase binding controls a whole recipe. These local observations narrow the design space without uniquely identifying a kernel or implementing an automatic selector.

The qualification sequence matters. Own-path shams test restoration fidelity, local crosses test aligned boundaries, and complete registered outputs test the candidate. B570's immediate null followed by trajectory changes makes the last step indispensable. Locality also does not guarantee low cost: mask 3 constrains fewer phases than mask 15, yet misses the extra generation-speed gate.

7.2. Use both paid baselines for operating decisions

Ordinary R/C measures a deployment change, matched M/C the increment from adding a draft. RTX's large ordinary-request improvement therefore includes a draft-free target-path opportunity; the remaining speculative increment is small and uneven. A faster qualified ordinary implementation could narrow the deployment benefit, since R is a paid reference rather than a proven fastest decoder.

The failures change concrete choices: retain the frozen RTX anchor rather than its slower qualified wider draft; reject the B570 recipe for latency against both strong baselines; avoid adding the measured M4 guard on an assumed control benefit; and address 358H target work or its state interface before expecting draft-only optimization to reach parity. These decisions use each campaign's contract rather than a shared platform ranking.

7.3. Scope of the evidence

Repair evidence covers one Qwen3 target/draft family, finite panels and pinned runtime variants. The RTX project-unused panel is a selected 24-input MBPP panel; unused means absent from accessible project execution records, not unseen during training. Geometry failures require renewed qualification after configuration changes. The results establish neither cross-family transfer nor reference preservation on arbitrary inputs.

Discrete output agreement is not numerical equivalence. The score/KV witnesses do not separate every reduction, precision, layout or scheduling effect; shams cannot exhaust hidden backend state. Selected cuts and a four-switch family cannot establish universal minimality or superiority over unimplemented prior methods.

Task medians and descriptive intervals do not eliminate session, thermal, ordering or infrastructure effects. B570's missing old-N1 payload and unknown raw comparisons remain explicit. Timers and historical energy domains differ, and later direct-ID qualification cannot certify earlier text-based costs. The author-side CPU reconstruction covers included sources and exports, with inherited private-archive audits identified separately (Supplement §S7).

8. Conclusion

Reference-preserving speculative execution requires a qualified transaction, not merely correct acceptance logic at one decision. The studied RTX implementation constrains attention query layout during target prefill and continuation while retaining other batched computation. Natural crosses and a four-switch qualification study support that finite repair scope; complete responses and width failures define its boundary. Paid ordinary and matched comparisons then separate the deployment gain from the speculative increment. The resulting decisions are mixed: the frozen RTX anchor helps on aggregate, a wider qualified draft loses, the B570 repair remains slower than strong baselines, and M4 and 358H restrict the case for added control and draft-only optimization. The study supplies concrete implementation evidence and reproducible cost comparisons within one model family, while preserving failures that prevent broader claims.

Author-review declarations

Authorship and contributions. Author names, affiliations, corresponding-author details and individual CRediT roles await author confirmation. This draft does not assign authorship or contributions.

Funding and competing interests. Funding sources and competing-interest declarations await confirmation by all authors; absence of a funding source or competing interest is not asserted.

Data and software availability. The author-review source package includes frozen committed evidence exports, source bindings, selected implementation and report records, and CPU-only code for recalculation and figures. Original runtime code and experiment records are retained in the HeteroSpec project. Availability and permitted redistribution of private raw archives, model artifacts and a permanent public data/software location require author confirmation before submission. The supplied exports enable the numerical checks specified in Supplement §S7; they do not substitute for a full private-archive audit or a rerun of model inference.

Declaration of generative AI and AI-assisted technologies in manuscript preparation. OpenAI ChatGPT/Codex was used to assist with restructuring and drafting text, source and bibliography checks, CPU analysis and plotting code, and the execution-architecture illustration. Quantitative figures are generated from the identified experimental exports, without generative replacement of observations. This author-review version still requires the authors' final scientific, bibliographic and language verification. The final declaration and responsibility statement must be confirmed by the authors before submission.

References

[1] Y. Leviathan, M. Kalman, Y. Matias, Fast inference from transformers via speculative decoding, in: Proceedings of the 40th International Conference on Machine Learning, Vol. 202, PMLR, 2023, pp. 19274–19286, accessed 9 October 2026.
URL `https://proceedings.mlr.press/v202/leviathan23a.html`

[2] ggml-org llama.cpp contributors, llama.cpp [software], Releases b10809 and b10839; private study patches included in the author source package, b10809 released 4 September 2026; b10839 released 7 September 2026. Accessed 9 October 2026 (2026).
URL https://github.com/ggml-org/llama.cpp/releases/tag/b10839

[3] A. Yang, et al., Qwen3 technical report, arXiv preprint arXiv:2505.09388v1, accessed 9 October 2026 (2025). doi:10.48550/arXiv.2505.09388.
URL https://arxiv.org/abs/2505.09388v1

[4] H. He, Thinking Machines Lab, Defeating nondeterminism in llm inference, Thinking Machines Lab: Connectionism, accessed 9 October 2026 (2025). doi:10.64434/tml.20250910.
URL https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/

[5] R. Gond, A. K. Kamath, R. Ramjee, A. Panwar, LLM-42: Enabling determinism in LLM inference with verified speculation, arXiv preprint arXiv:2601.17768v2, accessed 9 October 2026 (2026). doi:10.48550/arXiv.2601.17768.
URL https://arxiv.org/abs/2601.17768v2

[6] A. B. Lorup, Stage-replay divergence follows the KV cache: Fixed-prefix precision controls and bidirectional cache transplantation, arXiv preprint arXiv:2607.28495v1, accessed 9 October 2026 (2026). doi:10.48550/arXiv.2607.28495.
URL https://arxiv.org/abs/2607.28495v1

[7] J. Qin, A. Du, D. Zhang, M. Lentz, D. Zhuo, Vosti: Specifying, implementing, and verifying deterministic llm inference, arXiv preprint arXiv:2609.38981v1, accessed 9 October 2026; arXiv reports DOI registration pending (2026). doi:10.48550/arXiv.2609.38981.
URL https://arxiv.org/abs/2609.38981v1

[8] K. Chu, Y. Zhou, W. Zhang, MarginGate: Sparse margin-triggered verification for batch-invariant LLM inference, arXiv preprint arXiv:2605.30218v1, accessed 9 October 2026 (2026). doi:10.48550/arXiv.2605.30218.
URL https://arxiv.org/abs/2605.30218v1

[9] P. Chordiya, Lossless but not free: An empirical anatomy of speculative decoding on consumer hardware, arXiv preprint arXiv:2607.17283v1, accessed 9 October 2026 (2026). doi:10.48550/arXiv.2607.17283.
URL https://arxiv.org/abs/2607.17283v1

[10] X. Liu, J. Yu, J. Park, I. Stoica, A. Cheung, Speculative decoding: Performance or illusion?, in: Proceedings of Machine Learning and Systems, Vol. 8, 2026, accessed 9 October 2026.
URL https://proceedings.mlsys.org/paper_files/paper/2026/hash/554e056fe2b6d9fd27ffcd3367ae1267-Abstract-Conference.html

[11] L. Zhang, Z. Zhang, B. Xu, R. Li, Z. Tian, S. Mei, D. Li, Dovetail: A CPU/GPU heterogeneous speculative decoding for LLM inference, arXiv preprint arXiv:2412.18934v2, accessed 9 October 2026 (2025). doi:10.48550/arXiv.2412.18934.
URL https://arxiv.org/abs/2412.18934v2

[12] K. Lv, H. Guo, Q. Guo, X. Qiu, DuoDecoding: Hardware-aware heterogeneous speculative decoding with dynamic multi-sequence drafting, arXiv preprint arXiv:2503.00784v1, accessed 9 October 2026 (2025). doi:10.48550/arXiv.2503.00784.
URL https://arxiv.org/abs/2503.00784v1

[13] X. Liu, J. Park, L. Hu, W. Kwon, Z. Li, C. Zhang, K. Du, X. Mo, K. You, A. Cheung, Z. Deng, I. Stoica, H. Zhang, TurboSpec: Closed-loop speculation control system for optimizing LLM serving goodput, arXiv preprint arXiv:2406.14066v3, accessed 9 October 2026 (2025). doi:10.48550/arXiv.2406.14066.
URL https://arxiv.org/abs/2406.14066v3

[14] Y. Hou, F. Zhang, C. Du, X. Zhang, J. Pan, T. Pang, C. Du, V. Tan, Z. Yang, BanditSpec: Adaptive speculative decoding via bandit algorithms, in: Proceedings of the 42nd International Conference on Machine Learning, Vol. 267, PMLR, 2025, pp. 24045–24079, accessed 9 October 2026.
URL https://proceedings.mlr.press/v267/hou25h.html

[15] S. Liu, Y. Ye, Q. Zhu, Z. Cao, Y. He, HeteroSpec: Leveraging contextual heterogeneity for efficient speculative decoding, in: Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, 2026, pp. 12930–12947, accessed 9 October 2026. doi:10.18653/v1/2026.acl-long.589.
URL https://aclanthology.org/2026.acl-long.589/

[16] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. Ponde de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. Petroski Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. Hebgen Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, W. Zaremba, Evaluating large language models trained on code, arXiv preprint arXiv:2107.03374v2, accessed 9 October 2026 (2021). doi:10.48550/arXiv.2107.03374.
URL https://arxiv.org/abs/2107.03374v2

[17] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, C. Sutton, Program synthesis with large language models, arXiv preprint arXiv:2108.07732v1, accessed 9 October 2026 (2021). doi:10.48550/arXiv.2108.07732.
URL https://arxiv.org/abs/2108.07732v1

[18] H. Xia, Z. Yang, Q. Dong, P. Wang, Y. Li, T. Ge, T. Liu, W. Li, Z. Sui, Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding, in: Findings of the Association for Computational Linguistics: ACL 2024, Association for Computational Linguistics, 2024, pp. 7655–7671, accessed 9 October 2026. doi:10.18653/v1/2024.findings-acl.456.
URL https://aclanthology.org/2024.findings-acl.456/